

Manual de Uso

Gerador de Figuras — Modelo Eta 3km / BESM
SisMOM / CPTEC-INPE

Versao com configuracao YAML, parametro --run e ctl_to_yaml.py
Junho de 2026

1. Introducao

Este pacote de scripts Python gera figuras e arquivos GeoTIFF Cloud Optimized (COG) a partir das saidas binarias do modelo Eta 3km acoplado ao BESM, operado pelo SisMOM / CPTEC-INPE. A configuracao do ambiente e feita por arquivos YAML, sem necessidade de editar codigo Python.

1.1 Funcionalidades

- Leitura dos arquivos binarios GrADS (big-endian float32, formato TEMPLATE)
- Geracao de figuras PNG horarias para as 46 variaveis 2D do modelo
- Calculo e visualizacao de acumulados de precipitacao (24h sinoptico ou periodo configuravel)
- Exportacao de Cloud Optimized GeoTIFF (COG) compativeis com o SisMOM
- Processamento paralelo com multiprocessing (ProcessPoolExecutor)
- Configuracao completa via YAML: grade, modelo, caminhos, colormaps e limites
- Geracao automatica de config.yaml e variables.yaml a partir de CTL ou NetCDF (ctl_to_yaml.py)

1.2 Estrutura de arquivos

```
Figuras Eta/
config.yaml      # configuracao do modelo e caminhos
variables.yaml   # definicao das variaveis (colormaps, limites)
run.sh           # script de execucao com logging automatico
scripts/
  main.py        # orquestrador principal (CLI)
  config.py      # carregador YAML e constantes globais
  reader.py      # leitura dos binarios GrADS
  accumulate.py  # calculo e plot dos acumulados
  export_cog.py  # exportacao COG GeoTIFF
  plot_variables.py # geracao das figuras PNG
  plot_utils.py  # colormaps e utilitarios de plot
  ctl_to_yaml.py # gera config.yaml/variables.yaml a partir de CTL ou NetCDF
  download_data.py # download dos dados do SisMOM
  requirements.txt # dependencias Python
```

2. Instalacao

2.1 Requisitos

O sistema requer Python 3.8 ou superior. As dependencias estao listadas em scripts/requirements.txt:

Pacote	Versao minima	Finalidade
numpy	>= 1.24	Leitura e manipulacao dos arrays binarios
rasterio	>= 1.3	Exportacao COG GeoTIFF
pyyaml	>= 6.0	Leitura dos arquivos de configuracao YAML
matplotlib	>= 3.7	Geracao das figuras PNG (dispensavel em --cog-only)
netCDF4	>= 1.6	Leitura de NetCDF em ctl_to_yaml.py (opcional; ou scipy)

2.2 Instalacao das dependencias

O script run.sh instala automaticamente as dependencias se necessario. Para instalar manualmente:

```
pip install -r scripts/requirements.txt

# Para suporte a NetCDF em ctl_to_yaml.py:
pip install netCDF4      # recomendado
# ou: pip install scipy # alternativa
```

Nota: cartopy (linhas de costa) e opcional. Se disponivel, as figuras incluem contornos geograficos. Para instalar: conda install -c conda-forge cartopy

3. Configuracao

Toda a configuracao e feita em dois arquivos YAML na raiz do projeto. Nao e necessario editar nenhum arquivo Python. O script `ctl_to_yaml.py` (secao 9) pode gerar esses arquivos automaticamente a partir de um CTL ou NetCDF.

3.1 config.yaml

```
run:
  ntimes: 121          # numero de passos de tempo (inclui analise = t0)
  dt_hours: 1          # intervalo de saida do modelo em horas

grid:
  nx: 2399  ny: 1671
  lon0: -90.99252  lat0: -38.12884
  dlon: 0.03      dlat: 0.03

model:
  undef: 1.0e+20
  dtype: ">f4"      # big-endian float32
  file_prefix: "Eta03_BESM_{run_tag}+" # {run_tag} substituido pelo --run
  file_suffix: "_2D.bin"

paths:
  data_base: ""      # vazio = usar SISMOM_DATA_BASE ou data/
  output_dir: "figuras/campos"
  accum_dir: "figuras/acumulados"
  cog_dir: "cog"
  log_dir: "logs"

figure:
  ext: "png"  dpi: 120

accumulation:
  hours: 24      # 24h = ACUM00Z + ACUM12Z; outros = janelas sequenciais
  precip_vars: [PREC, PRCV, PRGE]
```

3.2 variables.yaml

Define todas as variaveis 2D do modelo na mesma ordem do arquivo CTL. Cada variavel tem:

Campo	Tipo	Descricao
name	string	Nome da variavel (igual ao CTL do GrADS)
description	string	Descricao legivel para titulos das figuras
units	string	Unidade de medida exibida no colorbar
cmap	string	Colormap matplotlib ou "precip" para paleta propria

vmin / vmax	float	Limites do colorbar (null = escala automatica por percentis)
precip	boolean	true se variavel de precipitacao (converte m->mm nos COGs)
enabled	boolean	false para desativar plot e COG (mantém posicao binaria)

A ordem das variaveis em variables.yaml deve ser identica a ordem no arquivo CTL. Variaveis com enabled: false sao ignoradas para plot/COG, mas continuam sendo lidas do binario para preservar o alinhamento das posicoes.

3.3 Localizacao dos dados

1. Argumento --data_dir (caminho completo diretamente para a pasta com os .bin)
2. Argumento --data_base ou variavel de ambiente SISMOM_DATA_BASE
3. Campo paths.data_base em config.yaml
4. Diretorio local data/ na raiz do projeto (fallback)

4. Tag da Rodada (--run)

A tag da rodada identifica o horario de inicio da previsao no formato YYYYMMDDHH. Ela e passada sempre via CLI e nunca fica armazenada no config.yaml.

4.1 Formatos aceitos

Formato	Exemplo	Resultado
YYYYMMDDHH (completo)	--run 2026060600	Rodada de 06/06/2026 00Z
HH (so a hora)	--run 00	00Z da data atual do sistema
H (1 digito)	--run 0	00Z da data atual do sistema
Variavel de ambiente	export RUN_TAG=2026060612	Equivalente a --run 2026060612

Quando apenas a hora e informada (ex: --run 12), a data e obtida automaticamente do sistema (datetime.date.today()). Util em scripts de operacao automatica agendados pelo cron.

4.2 Exemplos

```
./run.sh --run 2026060600 # rodada completa especificada
./run.sh --run 00         # 00Z de hoje (data do sistema)
./run.sh --run 12         # 12Z de hoje
export RUN_TAG=2026060600 && ./run.sh --cog-only
```

5. Referencia de Linha de Comando

5.1 Execucao via run.sh (recomendado)

```
./run.sh --run 00 # PNG + acumulados 24h
./run.sh --run 00 --cog-only # somente COG GeoTIFF
./run.sh --run 00 --cog --workers 8 # PNG + COG em paralelo
./run.sh --run 00 --accum-hours 6 # acumulado de 6h (sequencial)
./run.sh --run 00 --only-accum # somente acumulados
```

5.2 Todos os parametros

Parametro	Padrao	Descricao
--run TAG	—	Tag da rodada: YYYYMMDDHH ou HH. Obrigatorio
--config YAML	config.yaml	Arquivo de configuracao alternativo
--vars-file YAML	variables.yaml	Arquivo de variaveis alternativo
--data_base DIR	SISMOM_DATA_BASE	Base SisMOM; completa com /<run>/regional/eta/2D/
--data_dir DIR	—	Caminho completo para os .bin (sobrepoe --data_base)
--output_dir DIR	figuras/campos	Destino dos PNGs de campos horarios
--accum_dir DIR	figuras/acumulados	Destino dos PNGs de acumulados
--cog_dir DIR	cog	Destino dos COG GeoTIFF
--vars V1 V2 ...	enabled em YAML	Lista de variaveis a processar
--accum_hours N	24	Periodo do acumulado em horas
--workers N	ncpus	Processos paralelos
--only_accum	false	Gera somente acumulados
--only_fields	false	Gera somente campos horarios
--sequential	false	Leitura com marcadores Fortran
--cog	false	Exportar COG alem dos PNGs
--cog_only	false	Exportar somente COG (sem PNGs)
--cog_overviews	false	Embutir piramides nos COGs
--skip_existing	false	Pula arquivos ja gerados
--quiet	false	Suprime saida de progresso

6. Campos Acumulados

O sistema calcula campos acumulados subtraindo o valor do horario inicial do horario final. O nome do arquivo de saida inclui o periodo acumulado para facil identificacao.

6.1 Convencao de nomenclatura

```
# Formato geral:
VARNAME_YYYYMMDDHH_acumNNh.png

# Exemplos:
PREC_2026060600_acum24h.png # acumulado 24h, rodada 00Z
PREC_2026060600_acum06h.png # acumulado 6h, rodada 00Z
PRCV_2026060612_acum12h.png # convectiva, acumulado 12h
```

6.2 Periodo acumulado

O periodo e controlado pela opcao --accum-hours. O padrao e 24h. Qualquer divisor inteiro do total de horas de previsao e valido.

```
./run.sh --run 00 --accum-hours 24 # padrao: 00-24h, 24-48h, ...
./run.sh --run 00 --accum-hours 6  # janelas de 6h
./run.sh --run 00 --accum-hours 12 # janelas de 12h
./run.sh --run 00 --only-accum      # somente acumulados (sem campos horarios)
```

Variaveis de precipitacao (precip: true em variables.yaml) sao multiplicadas por 1000 na conversao m → mm ao exportar para COG GeoTIFF.

7. Exportacao COG GeoTIFF

Alem das figuras PNG, o sistema pode exportar cada campo como Cloud-Optimized GeoTIFF (COG), adequado para ingestao em sistemas GIS, servidores de tiles e pipelines de processamento.

7.1 Ativando a exportacao

```
./run.sh --run 00 --cog          # PNG + COG em paralelo
./run.sh --run 00 --cog-only     # somente COG (sem PNGs)
./run.sh --run 00 --cog --cog-overviews # COG com piramides embutidas
```

7.2 Configuracao no config.yaml

```
cog:
  enabled: true
  output_dir: cog
  compress: deflate      # deflate | lzw | zstd | none
  overviews: false      # true = embutir piramides (GDAL)
  resampling: nearest   # nearest | bilinear | cubic
  dtype: float32        # float32 | float64 | int16
```

7.3 Estrutura dos arquivos gerados

```
cog/
  2026060600/
    PREC_2026060600_F001.tif
    PREC_2026060600_F002.tif
    ...
    PREC_2026060600_acum24h.tif
    TP2M_2026060600_F001.tif
    ...
```

8. Download de Dados

O sistema suporta download automatico dos arquivos binarios do SisMOM antes do processamento.

8.1 Configuracao

```
download:
  enabled: true
  server: sismom.cptec.inpe.br
  protocol: sftp          # sftp | ftp | http
  base_path: /dados/eta/3km/
  timeout: 120            # segundos
  retries: 3
```

8.2 Uso

```
# Download + processamento em uma etapa:
./run.sh --run 00 --download

# Apenas download (sem gerar figuras):
./run.sh --run 00 --download-only
```

9. Geracao Automatica de Configuracao (ctl_to_yaml.py)

O script `ctl_to_yaml.py` le um descritor de dados — arquivo GrADS CTL ou NetCDF — e gera automaticamente os arquivos `config.yaml` e `variables.yaml`, eliminando a necessidade de escrita manual.

9.1 Dependencias

Biblioteca	Instalacao	Uso
netCDF4	<code>pip install netCDF4</code>	Leitura de arquivos NetCDF (preferencial)
scipy	<code>pip install scipy</code>	Fallback para netCDF3 se netCDF4 falhar
numpy	<code>pip install numpy</code>	Inferencia de dtype (big/little-endian)
PyYAML	<code>pip install pyyaml</code>	Nao utilizado (YAML gerado manualmente)

netCDF4 e opcional: se nao estiver instalado, o script ainda funciona com arquivos CTL. Para suporte a NetCDF3 classico use scipy como fallback automatico.

9.2 Uso basico

```
# A partir de um descritor CTL:
python scripts/ctl_to_yaml.py Eta03_BESM.ctl

# A partir de um arquivo NetCDF:
python scripts/ctl_to_yaml.py saida_modelo.nc

# Saida padrao: config.yaml e variables.yaml no diretorio atual
# Use --next-to-input para salvar junto ao arquivo descritor:
python scripts/ctl_to_yaml.py Eta03_BESM.ctl --next-to-input
```

9.3 Opcoes de linha de comando

Opcao	Padrao	Descricao
input (posicional)	—	Caminho para o CTL ou arquivo NetCDF. Obrigatorio
<code>--format auto ctl netcdf</code>	auto	Forca o formato; 'auto' detecta pela extensao
<code>--config ARQUIVO</code>	<code>config.yaml</code>	Nome/caminho do <code>config.yaml</code> gerado

--vars ARQUIVO	variables.yaml	Nome/caminho do variables.yaml gerado
--next-to-input	false	Salva os YAMLS no mesmo diretorio do input
--next-to-ctl	false	Alias de --next-to-input (retrocompatibilidade)
--stdout	false	Imprime os YAMLS no terminal, sem criar arquivos
--dry-run	false	Simula sem escrever arquivos (implica --stdout)
--list-vars	false	Lista as variaveis detectadas e encerra
--force	false	Sobrescreve arquivos existentes sem perguntar

9.4 Exemplos de uso

```
# Listar as variaveis de um NetCDF sem gerar arquivos:
python scripts/ctl_to_yaml.py saida.nc --list-vars

# Visualizar o YAML gerado sem salvar em disco:
python scripts/ctl_to_yaml.py Eta03.ctl --dry-run

# Especificar nomes de saida explicitamente:
python scripts/ctl_to_yaml.py saida.nc \
    --config /etc/figuras/config.yaml \
    --vars /etc/figuras/variables.yaml

# Forcar leitura como CTL (extensao incomum):
python scripts/ctl_to_yaml.py arquivo.des --format ctl

# Sobrescrever YAMLS existentes:
python scripts/ctl_to_yaml.py Eta03.ctl --force
```

9.5 Parsing de arquivos CTL

O parser CTL suporta a sintaxe padrao GrADS. Os campos lidos sao:

Diretiva CTL	Campo em config.yaml	Observacao
DSET	file_prefix / file_suffix	Extrai prefixo e sufixo do padrao de nome
UNDEF	undef_value	Valor de dado ausente (ex: -9.99e20)
OPTIONS BYTESWAPPED	dtype: '<f4'	Sem BYTESWAPPED usa '>f4' (big-endian)

XDEF N LEVELS	grid: nx, x0, dx	Numero de pontos, longitude inicial e incremento
YDEF N LEVELS	grid: ny, y0, dy	Numero de pontos, latitude inicial e incremento
TDEF	dt_hours, nt	Passo temporal e numero de passos
VARS/ENDVARS	variables:	Uma entrada por variavel, com niveis e descricao

9.6 Parsing de arquivos NetCDF

O parser NetCDF segue as convencoes CF (Climate and Forecast). A deteccao de coordenadas usa tres estrategias em cascata:

Prioridade	Estrategia	Exemplo
1 (maior)	Nome exato na lista de candidatos	lon, longitude, XLONG, nav_lon, x
2	Atributo 'axis' da variavel (CF)	axis='X', axis='Y', axis='T'
3	Atributo 'standard_name' (CF)	standard_name='longitude'

```
# Formatos de unidade de tempo suportados (CF conventions):
'hours since 2026-01-01 00:00:00' -> dt_hours a partir dos valores
'days since 1900-01-01'          -> dt convertido para horas
'minutes since ...'              -> dt convertido para horas

# Grids curvilineares (lat/lon 2D):
# A primeira linha/coluna e usada para extrair x0, dx, y0, dy
# (aproximacao valida para grids aproximadamente regulares)
```

9.7 Inferencia de colormap e limites

Para cada variavel detectada, o script infere automaticamente o colormap e os limites do colorbar usando uma tabela interna com ~60 nomes conhecidos do modelo Eta/BESM. Quando o nome nao e reconhecido, o fallback usa a unidade da variavel.

Critério	Exemplos de variaveis	Colormap inferido
Nome exato (tabela interna)	PREC, PRCV, PRGE, NEVE	precip (colormap proprio)
Nome exato	TP2M, TSFC, TSOIL, MXTP, MNTP	RdBu_r, hot_r, cool
Nome exato	U10M, V10M, U100, V100	bwr
Fallback por unidade	k, kelvin -> RdBu_r	RdBu_r
Fallback por unidade	hpa, pa -> RdBu_r	RdBu_r

Fallback por unidade	m/s, kt -> bwr	bwr
Fallback por unidade	mm/h, mm/d -> precip	precip
Padrao final	qualquer outro	viridis / escala automatica

10. Estrutura de Diretórios

```
figuras-eta/
├── run.sh                # ponto de entrada principal
├── config.yaml           # configuracao do sistema
├── variables.yaml        # definicao das variaveis
├── scripts/
│   ├── main.py           # orquestrador do pipeline
│   ├── reader.py         # leitura dos .bin do Eta
│   ├── plotter.py        # geracao dos PNG
│   ├── cog_exporter.py   # exportacao COG GeoTIFF
│   ├── accumulator.py    # calculo de acumulados
│   ├── downloader.py     # download de dados
│   └── ctl_to_yaml.py    # geracao de config a partir de CTL/NC
├── figuras/
│   ├── campos/           # PNGs horarios
│   └── acumulados/       # PNGs de acumulados
└── cog/                  # COG GeoTIFFs (se habilitado)
```

11. Solucao de Problemas

Sintoma	Causa provavel	Solucao
FileNotFoundError ao ler .bin	data_dir incorreto ou arquivo nao baixado	Verifique --data_dir ou habilite --download
Imagens completamente em branco	undef_value nao configurado	Verifique undef na secao data do config.yaml
ValueError: shape mismatch	nx/ny diverge do arquivo real	Confira XDEF/YDEF no CTL ou re-execute ctl_to_yaml.py
ModuleNotFoundError: netCDF4	biblioteca nao instalada	pip install netCDF4 (opcional, so para .nc)
PermissionError ao salvar COG	diretorio cog/ sem permissao de escrita	mkdir -p cog && chmod 755 cog
Paleta 'precip' nao encontrada	colormap personalizado nao registrado	Verifique scripts/colormaps.py e a importacao em plotter.py
ctl_to_yaml.py: SyntaxError YAML	caractere especial no descritor CTL	Abra o YAML gerado e corrija manualmente a linha indicada
Acumulado negativo	reinicio de contador no modelo (convectivo)	Normal para PRCV/PRGE; considere usar apenas PREC acumulado

12. Historico de Versoes

Versao	Data	Alteracoes
1.0	2025-01	Versao inicial: campos 2D, PNGs, acumulados 24h
1.1	2025-03	Suporte COG GeoTIFF, --cog-only, piramides opcionais
1.2	2025-06	Tag de rodada via CLI (--run), formato HH e YYYYMMDDHH
1.3	2025-09	Paralelismo (--workers), --skip-existing, --quiet
2.0	2026-05	Script ctl_to_yaml.py: geracao automatica de config.yaml e variables.yaml a partir de CTL
2.1	2026-06	Suporte NetCDF/CF em ctl_to_yaml.py: deteccao de coordenadas por nome, axis e standard_name